

在嵌入式裸机或 RTOS 开发中，链接脚本就像一张“内存地图”。如果某个段忘记指定 VMA（运行地址），链接器 ld 不会报错，而是自动把它紧挨着前一个段排布。结果可能轻则放错区域，重则直接导致系统崩溃。

【默认行为】

若段未指定 VMA/LMA：

VMA = 前一个输出段结束地址（顺延）

LMA = VMA（除非显式修改）

【可能的问题】

1. 段被放入错误的内存区域，例如本应进 Flash 却进了 RAM，导致 RAM 爆掉、变量覆盖、BSS/堆栈碰撞等。
2. 放入 MCU 不可访问的区域，引发硬fault（如 TCM、AXI SRAM、保留区等访问失败）。
3. 启动阶段 C 库初始化失败，static/global 对象构造函数找不到正确的 init_array 地址。
4. 多个段混杂在一起，导致内存布局不可预期。

【典型例子】

```
.my_special:
```

```
{
```

```
*(.myspecial)
```

```
}
```

若未指定 VMA，则 ld 可能把它贴在 .data 后面，结果只读配置被塞进 RAM。Flash 不会加载它，RAM 初始化也不会覆盖它，运行时读取到的都是垃圾数据，写入则破坏其他区域。

【总结】

忘记为段指定 VMA → ld 自动排放 → 但很可能错区 → 初始化异常 → 各类玄学崩溃。

在内存紧张的嵌入式系统中，必须确保每个段都有明确的“出生地”。